Construction of DAG Models for Autonomous Systems

Jing Huang[1], Kuan Jiang[1], Weijie Wang[2], Wei Liang[1], and Wanli Chang[2]
[1]School of Computer Science and Engineering, Hunan University of Science and Technology, China
[2]Key Laboratory for Embedded and Network Computing of Hunan Province, Hunan University, China
{huangjing@hnust.edu.cn, jkuan2022@163.com, weijiewang@hnu.edu.cn, wliang@hnust.edu.cn, wanli.chang.rts@gmail.com}

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11133396

***Abstract*—Directed Acyclic Graphs (DAGs) are widely deployed as task models in autonomous systems, including vehicles and drones, to capture functional dependency. DAG scheduling has been extensively investigated by various communities to shorten makespan, under the common assumption that the model itself is given a priori. This work studies a rarely touched problem — construction of DAG models — and considers time-triggered blended task chains predominant in autonomous systems. We report representation semantics and a topology optimization method. Experiments show that the average end-to-end response time reduction is 4.8 times of the conventional Floyd algorithm. Our time complexity is $\mathcal{O}(n^2)$, making it suitable for handling dynamic tasks as well.**

I. Introduction

Autonomous systems are increasingly applied in fields such as vehicles and drones. These systems typically need to perform a series of tasks in real-time to interpret the environment and make behavioral decisions. For instance, autonomous vehicles utilize multiple sensors to collect environmental data, and then process these data through a sequence of tasks such as data preprocessing and data fusion, which enable the identification of pedestrians, obstacles, and road signs in real-time. Beside the timely requirements, autonomous systems are usually resource-constrained. To enhance the system's real-time performance and resource utilization, much research has focused on task scheduling. However, the construction of task models is often overlooked. Most studies assume that task models and their dependencies are predefined, but in reality, the way tasks are modeled directly influences the effectiveness of scheduling. The design of task models, including how tasks and their relationships are abstracted, is crucial to the resulting performance of task execution.

Tasks in autonomous systems can be either time-triggered or event-triggered, but the majority are time-triggered, with each task having its own job release period. For example, in Autonomous Driving Systems (ADS), data sampling tasks are periodically triggered by various sensors, such as LiDAR and cameras [1]. Different tasks may be organized into one or more task chains to accomplish complex system functions [2]. Therefore, task dependencies must be carefully designed to enable their effective collaboration execution. Modeling these tasks as periodic Directed Acyclic Graphs (DAGs) facilitates task scheduling and behavior analysis, especially when tasks are synchronized over time, as DAGs effectively capture execution order and inter-task dependencies [3]. However, constructing a DAG that accurately reflects task dependencies and optimizes system performance presents significant challenges due to the complex data dependencies and varying periods of individual tasks.

We note that several methods have been proposed to address DAG modeling and conversion in autonomous systems. For instance, Verucchi et al. [4] introduced a method for converting a multi-rate DAG task set with time constraints into a single-rate DAG, aiming to optimize schedulability and end-to-end latency. Zhao et al. [3] focused on DAG scheduling and analysis by modeling parallelism and dependencies. Similarly, Sun et al. [5] proposed an integrated framework to analyze the schedulability of individual tasks and the end-to-end delay of task chains in a DAG. While these works consider DAG construction, they do not sufficiently address the execution efficiency of the resulting DAGs. Moreover, they emphasize using scheduling algorithms to enhance system performance.

In fact, purely relying on scheduling algorithms has limited potential to further improve time performance and resource utilization, as many algorithms are already approaching their optimal performance. However, approaching the problem from the perspective of task modeling may offer a new breakthrough. This paper delves into the construction of DAG models that accurately represents all jobs released by each task modules within a hyper-period, along with their dependencies. Our main contributions are as follows:

- A novel DAG construct approach is proposed. The approach first represents the jobs released by each task module on the chain within the hyper-period as vectors, and then apply the Kronecker product to obtain a matrix capturing all jobs and their relationships based on module dependencies, thereby constructing the initial DAG.
- A DAG simplification method, DRM, is reported. DRM can enhance the execution efficiency of the constructed DAG by reducing its redundant edges, leading to high average waiting times for jobs. Compared to the Floyd algorithm, which has a time complexity of $O(n^3)$, DRM

This work was supported by the National Key Research and Development Program of China (2023YFB4503704), the National Natural Science Fund Joint Key Project of China (U2468205), and the Hunan Natural Science Foundation (2023JJ30263). Corresponding author: Wanli Chang.

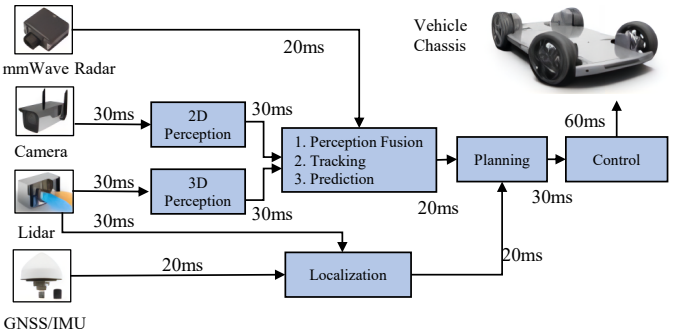

Fig. 1. An example of an ADAS (Advanced Driver Assistance System) in an intelligent vehicle

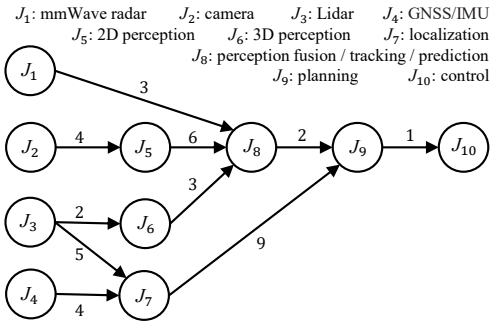

Fig. 2. Modeling the ADAS in Fig. 1, which includes 10 task modules and their dependencies.

achieves a lower time complexity of $O(n^2)$. Furthermore, to demonstrate that our method achieves a shorter average end-to-end response time, we used classic scheduling algorithms like HEFT and PEFT to schedule three types of DAGs: the unoptimized DAG, the DAG optimized by the Floyd algorithm, and our optimized DAG. Experimental results indicate that the Floyd algorithm-optimized DAG reduces the average end-to-end response time by about 2%, while our optimized DAG achieves a reduction of approximately 9.6%.

These contributions address existing limitations by providing a flexible, efficient task management framework that improves real-time responsiveness and resource allocation in autonomous systems, ultimately advancing their operational robustness.

II. Models and Problem Statement

A. Function Model

In autonomous systems, a function is often regarded as a task chain consisting of a series of task modules that periodically release jobs. These task modules can operate either sequentially or in parallel. Fig. 1 illustrates the ADAS (Advanced Driver Assistance System) in an intelligent vehicle, where modules such as Data Acquisition (DA), Data Perception (DP), Data Fusion (DF), Path Planning (PP), and Body Control (BC) form a task chain to collaboratively execute tasks. Each module has its own period; for example, the camera samples frames every 30ms, while the BC operates every 60ms. Clearly, this is an example of a directed acyclic graph of periodic tasks with data dependencies.

A function's major feature is reflected in the task modules it has and the relationships between them. Therefore, we model a function as: $F = \{J_1, J_2, \ldots, J_m, \boldsymbol{R}\}$. Here, $J_1, J_2, \ldots, J_m$ represent the task modules of the function, each defined as a tuple $J_i = (t_i, EC_i)$, where t_i denotes the time interval at which module J_i releases a job, and EC_i represents its execution cost. The term $\boldsymbol{R}$ represents the relationships between the m modules and is given by the following matrix:

$$\boldsymbol{R} = \begin{pmatrix} r_{1,1} & r_{1,2} & \cdots & r_{1,m} \\ r_{2,1} & r_{2,2} & \cdots & r_{2,m} \\ \vdots & \vdots & \ddots & \vdots \\ r_{m,1} & r_{m,2} & \cdots & r_{m,m} \end{pmatrix}. \quad (1)$$

- $r_{i,j} = 0$ represents that there is no dependency between modules J_i and J_j. Modules J_i and J_j can be executed in parallel.
- $r_{i,j} = 1$ indicates that module J_i is a direct predecessor of module J_j. In this case, an additional communication cost $C_{i,j}$ is often required to send the processing results of J_i to J_j.

Fig. 2 illustrates an example of a function model with 10 modules abstracted from the ADAS shown in Fig. 1. The related parameters of modules are listed in Table I.

TABLE I
Parameter settings for each module in Fig. 2

Modules	J_1	J_2	J_3	J_4	J_5	J_6	J_7	J_8	J_9	J_{10}
Job period	20	30	30	20	30	30	20	20	30	60
Execution cost	10	9	6	5	8	7	10	6	5	2

B. End-to-End Response Time Model

The jobs activated by module J_i within a hyper-period T can be denoted as the set $\boldsymbol{J}_i = \{J_{i,1}, J_{i,2}, \ldots, J_{i,T/t_i}\}$. Similarly to existing studies, we set $T = lcm(t_1, t_2, \ldots, t_m)$. Correspondingly, the set of all jobs can be expressed as $\boldsymbol{J} = \{\boldsymbol{J}_1 \cup \boldsymbol{J}_2 \cup \cdots \cup \boldsymbol{J}_m\}$. If $r_{i,j} = 1$, indicating that module J_i is the predecessor of J_j, it is essential to ensure that no data is lost during the transmission from the predecessor to the successor within the hyper-period T [6]. To achieve this, every job $J_{j,k'} \in \boldsymbol{J}_j$ must have a corresponding precursor $J_{i,k} \in \boldsymbol{J}_i$, and conversely, each $J_{i,k} \in \boldsymbol{J}_i$ must have a corresponding successor $J_{j,k'} \in \boldsymbol{J}_j$. For convenience, we denote the sets of predecessors and successors of $J_{i,k}$ as $pre(J_{i,k})$ and $suc(J_{i,k})$, respectively. A job with no predecessors is referred to as an entry job J_{entry}. Similarly, a job with no successors is referred to as an exit job J_{exit}. A function can have multiple entry jobs, but it usually produces only one output. Therefore, we assume that the function has a unique exit job J_{exit}, and use $\boldsymbol{J}_{\text{entry}}$ to represent the set all of entry jobs, $\boldsymbol{J}_{\text{entry}} = \{J_{i,k} \mid J_{i,k} \in \boldsymbol{J},\ |pre(J_{i,k})| = 0\}$.

The end-to-end response time refers to the time from the arrival of the entry job released by module J_i to the completion of the unique exit job J_{exit}, and is given by

$$RT(J_{i,k}) = FT(J_{\text{exit}}) - AT(J_{i,k}),\ J_{i,k} \in \boldsymbol{J}_{\text{entry}} \cap \boldsymbol{J}_i. \quad (2)$$

Here, $AT(J_{i,k})$ represents the arrival time of $J_{i,k}$, which can be obtained by adding an offset of $k-1$ periods to $AT(J_{i,1})$.

$$AT(J_{i,k}) = AT(J_{i,1}) + (k-1) \times t_i. \tag{3}$$

$FT(J_{\text{exit}})$ is the finish time of the exit job J_{exit}. Since the focus of this paper is on the construction of the model rather than the scheduling strategy, we directly use classic algorithms, such as HEFT [7] and PEFT [8], to calculate $FT(J_{\text{exit}})$.

Since each task generates a potentially infinite sequence of jobs, a schedulability interval is needed to capture the overall system behavior [6]. For this interval, Goossens et al. [9] proposed a calculation method. Within the hyper-period T, multiple entry jobs can be released. We calculate the average end-to-end response time for all entry jobs in $\boldsymbol{J}_{\text{entry}}$ as follows:

$$ART(\boldsymbol{J}_{\text{entry}}) = \frac{1}{|\boldsymbol{J}_{\text{entry}}|} \times \sum_{J_{\text{entry}} \in \boldsymbol{J}_{\text{entry}}} RT(J_{\text{entry}}). \tag{4}$$

C. Problem Statement

With the above models, our problem can be formulated as follows: Given a function $F = \{J_1, J_2, \ldots, J_m, \boldsymbol{R}\}$, along with the job release period t_i and execution cost EC_i for each task module J_i ($1 \le i \le m$), the goal is to construct a DAG that can capture the dependencies of the jobs $\boldsymbol{J}$ released from the function within a hyper-period $T = lcm(t_1, t_2, \ldots, t_m)$. The resulting DAG should minimize the average end-to-end response time of jobs in the function as much as possible when scheduled by existing algorithms.

III. DAG Construction and Optimization

Directed Acyclic Graph (DAG) modeling not only clearly represents job dependencies but also facilitates parallelism analysis.

A. DAG Construction

Each task module J_i actives a job at intervals t_i time. Within a given time T, the number of jobs activated by module J_i is T/t_i. For the function with m task modules, the number of activated jobs within time T is

$$n = \sum_{i=1}^{m} T/t_i. \tag{5}$$

To represent the relationships among the n jobs, we construct a DAG $\boldsymbol{G}$ and represent it as an adjacency matrix, denoted as:

$$\boldsymbol{G} = \begin{pmatrix} \boldsymbol{G}_{1,1} & \boldsymbol{G}_{1,2} & \ldots & \boldsymbol{G}_{1,m} \\ \boldsymbol{G}_{2,1} & \boldsymbol{G}_{2,2} & \ldots & \boldsymbol{G}_{2,m} \\ \vdots & \vdots & \ddots & \vdots \\ \boldsymbol{G}_{m,1} & \boldsymbol{G}_{m,2} & \ldots & \boldsymbol{G}_{m,m} \end{pmatrix}_{n \times n}. \tag{6}$$

Each element $\boldsymbol{G}_{i,j}$ is a sub-matrix of size $T/t_i \times T/t_j$, representing the dependency between the T/t_i jobs of module J_i and the T/t_j jobs of task module J_j.

It is evident that the value of $\boldsymbol{G}_{i,j}$ depends on the relationship between modules J_i and J_j. Generally, jobs within the same module are independent of each other, while the dependency between jobs from different modules is determined by the dependency between those modules. If module J_i is a direct predecessor of module J_j, i.e., $r_{i,j} = 1$, then any job in $\boldsymbol{J}_i$ may be a predecessor of each job in $\boldsymbol{J}_j$. Since the dependencies between modules are defined by the matrix $\boldsymbol{R}$, $\boldsymbol{G}_{i,j}$ can be calculated as follows:

$$\boldsymbol{G}_{i,j} = r_{i,j} \times \left(\mathbf{1}_{T/t_i \times 1} \otimes \mathbf{1}_{1 \times T/t_j}\right). \tag{7}$$

Here, $\mathbf{1}_{T/t_i \times 1}$ is a $T/t_i \times 1$ row vector with all elements equal to "1"; $\mathbf{1}_{1 \times T/t_j}$ is a $1 \times T/t_j$ column vector with all elements equal to "1"; the sign "$\otimes$" stands for the Kronecker product. Note that the sub-matrix $\boldsymbol{G}_{i,j}$ represents a temporarily constructed redundancy dependency between jobs, rather than an inherent characteristic of the jobs.

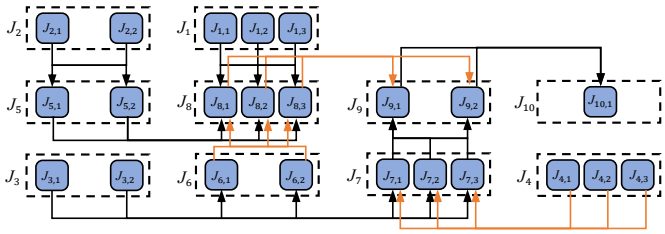

Fig. 3. The detailed structure of the DAG $\boldsymbol{G}$ with 23 jobs and 58 dependencies.

Take the ADAS in Fig. 1 as an example. Given time $T = 60$, we can construct a DAG $\boldsymbol{G}$ with 23 jobs and 58 dependencies, as shown in Fig. 3. To distinguish the different dependencies among jobs, we use different colors to label them in Fig. 3.

B. DAG Structure Optimization

Although we can use Eqs. (6) and (7) to construct a DAG representing the desired function, the resulting DAG is complex due to the intricate interconnections and hierarchical structure, complicating scheduling and parallel processing. Therefore, optimizing the DAG structure is crucial to enhance parallelism and execution efficiency.

1) Lower Bound Analysis of Dependencies: The DAG structure can be simplified by deleting redundant dependencies between jobs. According to ahead definition, the dependencies between jobs $\boldsymbol{J}_i$ and $\boldsymbol{J}_j$ are captured by $\boldsymbol{G}_{i,j}$. For convenience, we represent the element of $\boldsymbol{G}_{i,j}$ as $g_{k,k'}$, $g_{k,k'} \in \{0, 1\}$. In a real function, each task module transmits its processed results to the subsequent modules. To ensure the normal execution of the function, the following conditions for each job $J_{i,k} \in \boldsymbol{J}$ must be satisfied while optimizing the structure of the DAG $\boldsymbol{G}$.

- Satisfy the reachability of each job to the unique exit job;
- If $r_{i,j} = 1$, each job $J_{i,k} \in \boldsymbol{J}_i$ has at least one successor $J_{j,k'}$ released from module J_j, that is,

$$\sum_{k'=1}^{T/t_j} g_{k,k'} \ge 1; \tag{8}$$

- If $r_{i,j} = 1$, each job $J_{j,k'} \in \boldsymbol{J}_j$ has at least one predecessor $J_{i,k}$ released from module J_i, i.e,

$$\sum_{k=1}^{T/t_i} g_{k,k'} \ge 1. \tag{9}$$

Eqs. (8) and (9) ensure that each job can receive data from its predecessors and transmit its output to its successors.

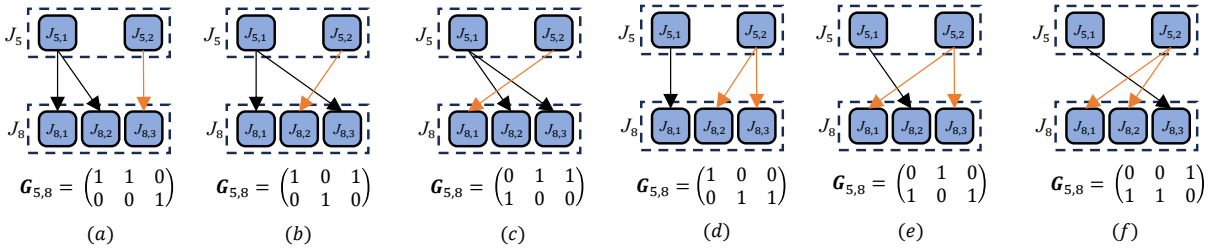

Fig. 4. 6 simple structures for sub-matrix $\boldsymbol{G}_{5,8}$ to improve parallelism and execution efficiency.

Theorem 1. *The minimum number of dependencies of DAG* $\boldsymbol{G}$ *is* $\sum_{i=1}^{m}\sum_{j=1}^{m} r_{i,j} \times max\{T/t_i, T/t_j\}$.

Proof: For each sub-matrix $\boldsymbol{G}_{i,j}$, when $t_i \le t_j$ (or $t_i > t_j$), setting $\sum_{k'=1}^{T/t_j} g_{k,k'} = 1$ (or $\sum_{k=1}^{T/t_i} g_{k,k'} = 1$) for each job $J_{i,k}$ (or $J_{j,k'}$), we can obtain that $\sum_{k=1}^{T/t_i}\sum_{k'=1}^{T/t_j} g_{k,k'} = \max\{T/t_i, T/t_j\}$. This setting ensures that both jobs $J_{i,k}$ and $J_{j,k'}$ satisfy the constraints of Eqs. (8) and (9), respectively. The same goes for other sub-matrices. ∎

Taking $\boldsymbol{G}_{5,8}$ in Eq. (6) as an example, $t_5 = 30$, $t_8 = 20$ and $T = 60$, so the minimum number of dependencies in sub-matrix $\boldsymbol{G}_{5,8}$ is $\max\{60/30, 60/20\} = 3$. Then, the minimum number of dependencies in matrix $\boldsymbol{G}$ in Fig. 3 is 27. Theorem 1 provides the minimum number of dependencies for the DAG, but identifying which redundant dependencies to delete requires further exploration. While there are various ways to remove redundant dependencies, our goal is to enhance the parallelism and execution efficiency of the DAG through their removal.

2) Removal of Dependencies: In DAG task scheduling, parallelism refers to the number of tasks that can be started simultaneously without violating dependencies. Therefore, a DAG with fewer dependencies has a relatively high parallelism. From Eq. (7), it follows that $\boldsymbol{G}_{i,j}$ is a $T/t_i \times T/t_j$ matrix, with all elements equal to "1" when $r_{i,j} = 1$. According to Theorem 1, the number of "1" in $\boldsymbol{G}_{i,j}$ can be reduced to $\max\{T/t_i, T/t_j\}$. In other words, $T/t_i \times T/t_j - \max\{T/t_i, T/t_j\}$ dependencies can be removed from $\boldsymbol{G}_{i,j}$ without compromising the correctness of the DAG.

To determine which dependencies can be removed, we assume the simplified $\boldsymbol{G}'_{i,j}$ as

$$\boldsymbol{G}'_{i,j} = \begin{pmatrix} g_{1,1} & g_{1,2} & \cdots & g_{1,T/t_j} \\ g_{2,1} & g_{2,2} & \cdots & g_{2,T/t_j} \\ \vdots & \vdots & \ddots & \vdots \\ g_{T/t_i,1} & g_{T/t_i,2} & \cdots & g_{T/t_i,T/t_j} \end{pmatrix}. \tag{10}$$

Based on Eq. (10), the problem of removal of dependencies becomes to solve the following equation.

$$\begin{cases} \sum_{k=1}^{T/t_i}\sum_{k'=1}^{T/t_j} g_{k,k'} = \max\left\{\frac{T}{t_i}, \frac{T}{t_j}\right\}, g_{k,k'} \in \{0,1\}; & \text{(I)} \\ g_{k,1} + \cdots + g_{k,\frac{T}{t_j}} \ge 1,\ \forall k, 1 \le k \le \frac{T}{t_i}; & \text{(II)} \\ g_{1,k'} + \cdots + g_{\frac{T}{t_i},k'} \ge 1,\ \forall k', 1 \le k' \le \frac{T}{t_j}. & \text{(III)} \end{cases} \tag{11}$$

It is obvious that Eq. (11) has multiple solutions.

Theorem 2. *The number of solutions that satisfy Eq. (11) is*

$$N = \binom{ab}{c} - a\binom{(a-1)b}{c} - b\binom{a(b-1)}{c} + ab\binom{(a-1)(b-1)}{c}, \tag{12}$$

where $a = T/t_i$, $b = T/t_j$, *and* $c = max\{a, b\}$.

Proof: The total number of choices to place "1" in c positions in the matrix $\boldsymbol{G}'_{i,j}$ is $N_{(\mathrm{I})} = \binom{ab}{c}$, satisfying condition (I) in Eq. (11). Condition (II) indicates that each row at least has one "1", which is similarly to Condition (III) corresponding to each column. The number of options that fail to meet them are $N_{(\mathrm{II})} = a\binom{(a-1)b}{c}$ and $N_{(\mathrm{III})} = c\binom{a(b-1)}{c}$, respectively. The case that both a row and a column have no "1" has $N_{(\mathrm{II})\&(\mathrm{III})} = ab\binom{(a-1)(b-1)}{c}$ options. Therefore, the number of solutions satisfying Eq. (11) is given by $N = N_{(\mathrm{I})} - N_{(\mathrm{II})} - N_{(\mathrm{III})} + N_{(\mathrm{II})\&(\mathrm{III})}$. ∎

Each solution corresponds to a specific structure of the DAG $\boldsymbol{G}$. Although these structures have the same number of dependencies, they may have difference execution efficiency. For sub-matrix $\boldsymbol{G}_{i,j}$, to demonstrate the impact of different structures on execution efficiency, we consider the Average Waiting Time (*AWT*) of jobs $\boldsymbol{J}_j$, which is defined as

$$AWT(\boldsymbol{J}_j) = \frac{1}{|\boldsymbol{J}_j|} \times \sum_{J_{j,k'} \in \boldsymbol{J}_j} WT(J_{j,k'}), \tag{13}$$

where $WT(J_{j,k'})$ represent the waiting time of job $J_{j,k'}$, measured from their release time to their start time, and is defined as

$$WT(J_{j,k'}) = ST(J_{j,k'}) - (k'-1) \times t_j. \tag{14}$$

If $g_{k,k'} = 1$ in $\boldsymbol{G}_{i,j}$, the start time of job $J_{j,k'}$ is given by

$$ST(J_{j,k'}) = ST(J_{i,k}) + EC_i + C_{i,j}. \tag{15}$$

Note that the start time in Eq. (14) does not take the availability of processors into account, as specific scheduling strategies are not employed and processor contention is not considered. Therefore, the start time of job $J_{i,k} \in \boldsymbol{J}_i$ can be considered as its release time, $ST(J_{i,k}) = (k-1) \times t_i$.

Illustration. We use $\boldsymbol{G}_{5,8}$, which illustrates the dependencies between jobs $\boldsymbol{J}_5$ and $\boldsymbol{J}_8$, as an example. According to Theorem 2, the number of all solutions of $\boldsymbol{G}_{5,8}$ that satisfy Eq. (11) is calculated as follows, $N_{\boldsymbol{G}_{5,8}} = \binom{6}{3} - 2\binom{3}{3} - 3\binom{4}{3} + 6\binom{2}{3} = 6$. Fig. 4 shows 6 possible structures ($a \sim f$) for $\boldsymbol{G}_{5,8}$.

Algorithm 1 Dependencies Removal Method (DRM)

Input: $\boldsymbol{G}$, m, T
Output: The simplified DAG $\boldsymbol{G}$

1: **for** i from 1 to m **do**
2: **for** j from 1 to m **do**
3: **if** $T/t_i \geq T/t_j$ **then** $\boldsymbol{G}_{i,j} = \boldsymbol{G}_{i,j}^{\mathrm{T}}$;
4: $N_{\mathrm{r}} = \min\{T/t_i, T/t_j\}$; $N_{\mathrm{c}} = \max\{T/t_i, T/t_j\}$;
5: **if** $N_{\mathrm{r}} > 1$ **then**
6: **for** k from N_{r} to 2 **do**
7: *minWT* = $+\infty$; // The minimal waiting time.
8: **for** k' from N_{c} to 1 **do**
9: $wt = WT(J_{j,k'})$ // using Eq. (14)
10: $minWT = wt < minWT \ ? \ wt : minWT$;
11: **end for**
12: Remove the dependencies from the sub-matrix $\boldsymbol{G}_{i,j}$ with $WT(J_{j,k'}) > minWT$;
13: **end for**
14: **end if**
15: **if** $T/t_i \geq T/t_j$ **then** $\boldsymbol{G}_{i,j} = \boldsymbol{G}_{i,j}^{\mathrm{T}}$;
16: **end for**
17: **end for**

Some parameters are given in the functionality model: $t_5 = 30$, $t_8 = 20$, $ET_8 = 8$, and $C_{5,8} = 6$. For structure *(a)* in Fig. 4,

$$AWT(\boldsymbol{J}_8) = \big((8+6)+(20-20)+(30+8+6-40)\big)/3 = 6.$$

The average waiting times for the other five structures are 12.33, 14.67, 14.0, 16.0, and 22.67, respectively. Obviously, structure *(a)* in Fig. 4 accelerates the start of each job in $\boldsymbol{J}_8$, resulting in higher execution efficiency compared to the others.

Algorithm description. With the execution efficiency in mind, we design the DRM (Dependencies Removal Method) approach as shown in Algorithm 1. The DRM optimizes each sub-matrix $\boldsymbol{G}_{i,j}$, and their operations are concentrated in lines 5-13. Lines 7-12 remove redundant dependencies with large waiting time between jobs $J_{i,k}$ and $\boldsymbol{J}_j$. By calling DRM for each sub-matrix, we obtain the simplified DAG $\boldsymbol{G}$. Fig. 5 shows the simplified structure of the DAG $\boldsymbol{G}$ in Fig. 3, consisting of 23 jobs and 27 dependencies. Compared to the original DAG, 31 dependencies have been removed.

Time complexity analysis. The most computationally intensive operation occurs in Lines 9 and 10, iterating through all elements of the matrix $\boldsymbol{G}$. Lines 9 and 10 require a total execution count given by $N_{et} = \sum_{i=1}^{m}\sum_{k=1}^{T/t_i}\sum_{j=1}^{m}\sum_{k'=1}^{T/t_j} 1$. According to Eq. (5), the number of jobs is $n = \sum_{i=1}^{m} T/t_i$. Obviously, we have $N_{et} = n \times n$. Therefore, the time complexity of the DRM algorithm is equal to $\mathcal{O}(n^2)$. Since DRM only traverses the edges of the DAG, its time complexity is lower compared to other optimization algorithms. The time complexity of the Floyd-Warshall algorithm is $\mathcal{O}(n^3)$, the Dijkstra algorithm (multi-source) is $\mathcal{O}(n^2(\log n + n))$, and the Bellman-Ford algorithm (multi-source) is $\mathcal{O}(n^4)$.

IV. Experimental Results and Discussion

A. Experiment Setup

To validate the effectiveness of our task modeling and optimization approach, we conduct extensive simulation experiments and compare it with *Floyd* [10]. All experiments are simulated using Python on a computer with an 11th Gen Intel(R) Core(TM) i7-11800H CPU at 2.30 GHz and 32.0 GB of RAM.

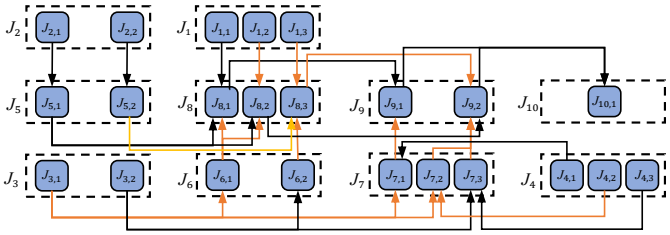

Fig. 5. A simplified structure of the DAG $\boldsymbol{G}$ in Fig. 3 with 23 jobs and 27 dependencies.

*1) **Workload Settings**:* The proposed DMR algorithm constructs the DAG based on the constraints between task modules, job periods, and loads. In our experiments, these parameters for task modeling are set as follows:

- *Number of modules.* $m \in \{10, 15, 20, 25\}$.
- *Module properties.* Each module J_i will active a job with execution cost EC_i at intervals t_i time, where $EC_i \in [1, 10]$ms and $t_i \in \{10, 25, 50\}$ms. The communication cost between modules J_i and J_j is set to $C_{i,j} \in [1, 4]$ms, $1 \leq i, j \leq m$.
- *Dependencies between modules.* We utilize a dependency generation tool sourced from the open-source project in [11] to simulate the relationship matrix $\boldsymbol{R}$ between modules. In this project, it is necessary to specify the number of modules, the in-degree and out-degree for each module (chosen from the set $\{1, 2, 3, 4, 5\}$), and the probability of each dependency being added (chosen from the set $\{0.3, 0.4, 0.5, 0.6\}$).

*2) **Platform Configuration**:* We consider computing platforms containing 8 identical processors.

*3) **Comparison Metrics**:* This section introduces two comparison metrics used in the experiments, as follow:

- *Average Waiting Time* (*AWT*). Refer to Eq. (13).
- *Average end-to-end Response Time* (*ART*). An important indicator of system execution efficiency.

B. Effectiveness of Task Modeling and Optimization

According to our workload settings, we will first construct numerous DAGs. Next, we shall simplify these DAGs using our approach DRM and the Floyd algorithm, respectively. Finally, we schedule both the simplified and unsimplified DAGs using the well-known HEFT and PEFT algorithms and compare their respective results. In our experiments, 1000 DAGs are synthesized with different numbers of modules.

*1) **Experiment 1**:* The results of scheduling using HEFT are shown in Fig. 6. We observe that the unsimplified DAGs have a higher *AWT* compared to the simplified cases using Floyd and DRM. Our proposed DRM achieves the smallest *AWT* compared with the unsimplified and Floyd. The specific *AWT* values for the unsimplified, Floyd, and DRM are 53.43ms, 50.18ms, and 42.52ms, respectively. Compared

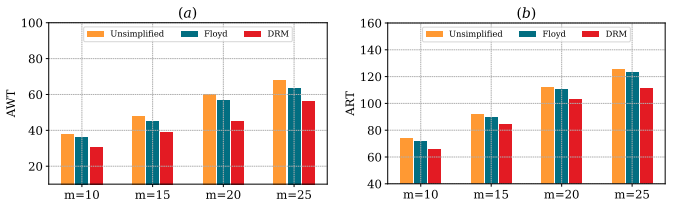

Fig. 6. Results of *AWT* and *ART* for the unsimplified, Floyd, and DRM, obtained by HEFT.

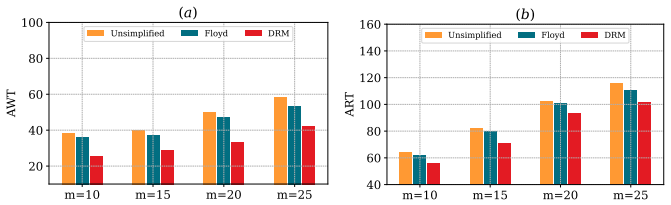

Fig. 7. Results of *AWT* and *ART* for the unsimplified, Floyd, and DRM, obtained by PEFT.

with unsimplified, Floyd and DRM reduce *AWT* by 6.08% and 20.42%, respectively. Clearly, Floyd has a smaller *AWT* compared to the unsimplified case, and our proposed DRM is smaller than Floyd. Through calculation, the *ART* values of the unsimplified, Floyd, and DRM are 101.04ms, 99.05ms, and 91.31ms, respectively. Compared with unsimplified, Floyd and DRM reduce *ART* by 1.97% and 9.63%, respectively. In summary, the relationship between the *AWT* and *ART* values for the unsimplified, Floyd, and DRM methods can be summarized as follows:

$$\begin{cases} AWT_{\text{Unsimplified}} > AWT_{\text{Floyd}} > AWT_{\text{DRM}}, \\ ART_{\text{Unsimplified}} > ART_{\text{Floyd}} > ART_{\text{DRM}}. \end{cases} \tag{16}$$

Therefore, we can be concluded that reducing the *AWT* of jobs can reduce the end-to-end response time to some extent. This also indicates that considering the *AWT* of jobs during the DAG structural optimization process is advisable.

*2) **Experiment 2**:* The results of scheduling using PEFT are shown in Fig. 7. The specific *AWT* values for the unsimplified, Floyd, and DRM are 46.43ms, 43.18ms, and 32.27ms, respectively. Compared with the *AWT* values in ***Experiment 1***, the *AWT* values obtained by PEFT are smaller than those of HEFT, which is attributed to the scheduling advantages of PEFT over HEFT. The *ART* values of the unsimplified, Floyd, and DRM are 90.03ms, 88.30ms, and 80.30ms, respectively. We can conclude that Eq. (16) holds true using PEFT. Compared to the unsimplified , Floyd and DRM reduce the *AWT* (*ART*) by 7.0% (1.92%) and 30.5% (10.81%), respectively.

The above two sets of experiments further validate the importance of optimizing the DAG structure and the rationale for considering the average waiting time of jobs during the structure optimization process.

V. Conclusion

In this paper, we have investigated the critical role of DAG construction in enhancing the real-time performance and resource utilization of autonomous systems, particularly in environments that have required intelligent decision-making across multiple task modules. We have introduced a novel approach to DAG abstraction and simplification that has considered task dependencies within a hyper-period, emphasizing both accurate representation and improved parallelism.

The proposed method has leveraged the Kronecker product to construct an initial DAG and has introduced an innovative edge simplification technique that has reduced average job waiting time while maintaining computational efficiency. Compared to traditional methods such as the Floyd algorithm, our approach has achieved a significant reduction in time complexity from $\mathcal{O}(n^3)$ to $\mathcal{O}(n^2)$ and has delivered superior performance in reducing the average response time of task chains. Experimental results have confirmed that the DAG optimized using our method has reduced the average response time by approximately 9.6%, significantly outperforming the 2% improvement achieved by the Floyd algorithm.

This work has highlighted the importance of DAG construction as a foundational step in performance optimization through scheduling strategy and has laid the groundwork for future research into real-time task management in autonomous systems. Potential future directions include extending the proposed methods to dynamic task environments and exploring integration with adaptive scheduling algorithms for enhanced robustness and scalability.

References

[1] M. Becker, D. Dasari, S. Mubeen, M. Behnam, and T. Nolte, "Synthesizing job-level dependencies for automotive multi-rate effect chains," in *2016 IEEE 22nd International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA)*, pp. 159–169, IEEE, 2016.

[2] M. Dürr, G. V. D. Brüggen, K.-H. Chen, and J.-J. Chen, "End-to-end timing analysis of sporadic cause-effect chains in distributed systems," *ACM Transactions on Embedded Computing Systems (TECS)*, vol. 18, no. 5s, pp. 1–24, 2019.

[3] S. Zhao, X. Dai, and I. Bate, "Dag scheduling and analysis on multi-core systems by modelling parallelism and dependency," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 12, pp. 4019–4038, 2022.

[4] M. Verucchi, M. Theile, M. Caccamo, and M. Bertogna, "Latency-aware generation of single-rate dags from multi-rate task sets," in *2020 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, pp. 226–238, IEEE, 2020.

[5] J. Sun, K. Duan, X. Li, N. Guan, Z. Guo, Q. Deng, and G. Tan, "Real-time scheduling of autonomous driving system with guaranteed timing correctness," in *2023 IEEE 29th Real-Time and Embedded Technology and Applications Symposium (RTAS)*, pp. 185–197, IEEE, 2023.

[6] J. Chen, C. Du, P. Han, and X. Du, "Work-in-progress: Non-preemptive scheduling of periodic tasks with data dependency upon heterogeneous multiprocessor platforms," in *2019 IEEE Real-Time Systems Symposium (RTSS)*, pp. 540–543, IEEE, 2019.

[7] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Transactions on Parallel and Distributed Systems*, vol. 13, no. 3, pp. 260–274, 2002.

[8] H. Arabnejad and J. G. Barbosa, "List scheduling algorithm for heterogeneous systems by an optimistic cost table," *IEEE Transactions on Parallel and Distributed Systems*, vol. 25, no. 3, pp. 682–694, 2014.

[9] J. Goossens, E. Grolleau, and L. Cucu-Grosjean, "Periodicity of real-time schedules for dependent periodic tasks on identical multiprocessor platforms," *Real-time systems*, vol. 52, pp. 808–832, 2016.

[10] A. E. Mirino *et al.*, "Best routes selection using dijkstra and floyd-warshall algorithm," in *2017 11th International Conference on Information & Communication Technology and System (ICTS)*, pp. 155–158, IEEE, 2017.

[11] https://github.com/laser/random-dag-generator-go, 2023.